Backprop como Functor: Uma perspectiva composicional sobre aprendizagem supervisionada

Brendan Fong David Spivak Rémy Tuyeras

Departamento de Matemática, Ciência da Computação e Laboratório de Inteligência Artificial, Instituto de Tecnologia de Massachusetts Instituto de Tecnologia de Massachusetts

arXiv:1711.10455v3 [math.CT] 1 de maio de 2019

Resumo — Um algoritmo de aprendizado supervisionado pesquisa um conjunto de funções $A \to B$ parametrizadas por um espaço P para encontrar a melhor aproximação para alguma função ideal $f : A \to B$. Ele faz isso tomando exemplos $(a, f(a)) \in A \times B$ e atualizando o parâmetro de acordo com alguma regra. Definimos uma categoria onde essas regras de atualização podem ser compostas e mostramos que a descida do gradiente — com relação a um tamanho de passo fixo e uma função de erro que satisfaz uma certa propriedade — define um functor monoidal de uma categoria de funções parametrizadas para essa categoria de regras de atualização. Uma contribuição fundamental é a noção de função de solicitação. Isso fornece uma perspectiva estrutural sobre retropropagação, proporcionando uma ampla generalização de redes neurais e vinculando-a com estruturas de programação bidirecional e jogos abertos.

I. Introdução

O aprendizado de máquina, e em particular o uso de redes neurais, tornou-se rapidamente notavelmente eficaz em tarefas do mundo real [18]. Um fator significativo para esse sucesso foi o algoritmo de retropropagação. A retropropagação permite calcular a derivada de uma função por meio da transmissão de mensagens em uma rede, acelerando significativamente o aprendizado. No entanto, embora o poder dessa abordagem seja impressionante, ela também é um tanto misteriosa. Quais estruturas tornam a retropropagação tão eficaz e como podemos interpretá-la, prevê-la e generalizá-la?

Nos últimos anos, categorias monoidais têm sido utilizadas para formalizar o uso de redes em computação e raciocínio — entre outras, as aplicações incluem diagramas de circuitos, processos de Markov, computação quântica e sistemas dinâmicos [8], [1], [6], [19]. Este artigo responde à necessidade de abordagens mais estruturais para o aprendizado de máquina, utilizando categorias para fornecer uma perspectiva algébrica e composicional sobre algoritmos de aprendizado e retropropagação.

Agradecemos a Patrick Schultz e Amalie Trewartha pelas discussões úteis. Trabalho apoiado por AFOSR FA9550-14-1-0031 e FA9550-17-1-0058.

978-1-7281-3608-0/19/$31,00 c©2019 IEEE

Considere um algoritmo de aprendizado supervisionado. O objetivo de um algoritmo de aprendizado supervisionado é encontrar uma aproximação adequada para uma função $f : A \to B$. Para fazer isso, o supervisor fornece uma lista de pares $(a, b) \in A \times B$, cada um dos quais deve aproximar os valores tomados por f, ou seja, $b \approx f(a)$. O supervisor também define um espaço de funções sobre o qual o algoritmo de aprendizado pesquisará. Isso é formalizado escolhendo um conjunto P e uma função $I : P \times A \to B$. Denotamos a função no parâmetro $p \in P$ como $I(p, -) : A \to B$. Então, dado um par $(a, b) \in A \times B$, o algoritmo de aprendizado pega uma aproximação hipotética atual de f, digamos dada por $I(p, -)$, e tenta melhorá-la, retornando alguma nova melhor estimativa, $I(p, -)$. Em outras palavras, um algoritmo de aprendizado supervisionado inclui uma função de atualização $U : P \times A \times B \to P$ para I.

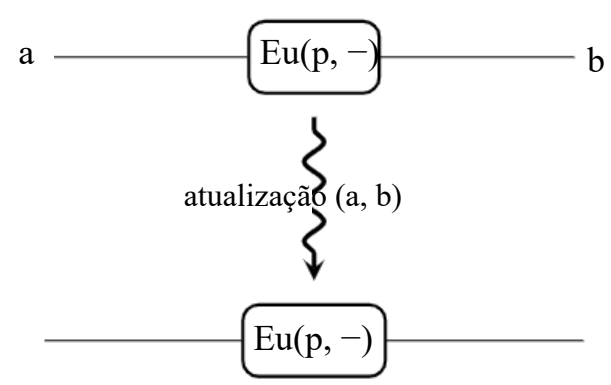


Figura 1. Dado um dado de treinamento (a, b), um algoritmo de aprendizagem atualiza p para p.

Para tornar isso composicional, fazemos a seguinte pergunta. Suponha que nos sejam fornecidos dois algoritmos de aprendizagem, conforme descrito acima, um para aproximar funções $A \to B$ e outro para funções $B \to C$. Podemos juntá-los para criar um algoritmo de aprendizagem para aproximar funções $A \to C$? Veremos que a resposta é não, porque algo está faltando.

Para construir um algoritmo de aprendizagem para o composto, precisaríamos de uma função parametrizada $A \to C$, bem como de uma regra de atualização. É fácil tomar as funções parametrizadas dadas $I : P \times A \to B$ e $J : Q \times B \to C$ e produzir uma de A a C. De fato, tome $P \times Q$ como

espaço de parâmetros e definir a função parametrizada $P \times Q \times A \to C$; $(p, q, a) 7 \to J(q, I(p, a))$. Chamamos a função $J(-, I(-, -)) : P \times Q \times A \to C$ de função parametrizada composta.

O problema está na definição da regra de atualização para o aprendiz composto. Nosso algoritmo deve tomar como dados de treinamento os pares (a, c) em $A \times C$. No entanto, para usar as funções de atualização fornecidas, escritas U e V para atualizar I e J, respectivamente, precisamos produzir dados de treinamento da forma (a, b) em $A \times B$ e (b, c) em $B \times C$. É simples produzir um par em $B \times C$ — tome

$$(\quad\quad)$$

—mas não há um par natural (a, b) para usar como dados de treinamento para I. Em (p, a),

A escolha de bshould codifica algo sobre as informações em c e J, e nada disso foi especificado.

Assim, para completar o quadro composicional, precisamos adicionar ao nosso formalismo uma maneira de o segundo algoritmo de aprendizagem repassar elementos de B. Chamaremos isso de função de solicitação, pois é como se J estivesse dizendo a I qual entrada b teria sido mais útil. A função de solicitação para J terá a forma $s: Q \times B \times C \to B$: dada uma hipótese q e dados de treinamento (b, c), ela retorna $b \coloneqq s(q, b, c)$. Agora, temos os dados de treinamento desejados (a, b) para I. A função de solicitação é, portanto, uma maneira de "retropropagar" a saída de volta para os aprendizes anteriores em uma rede.

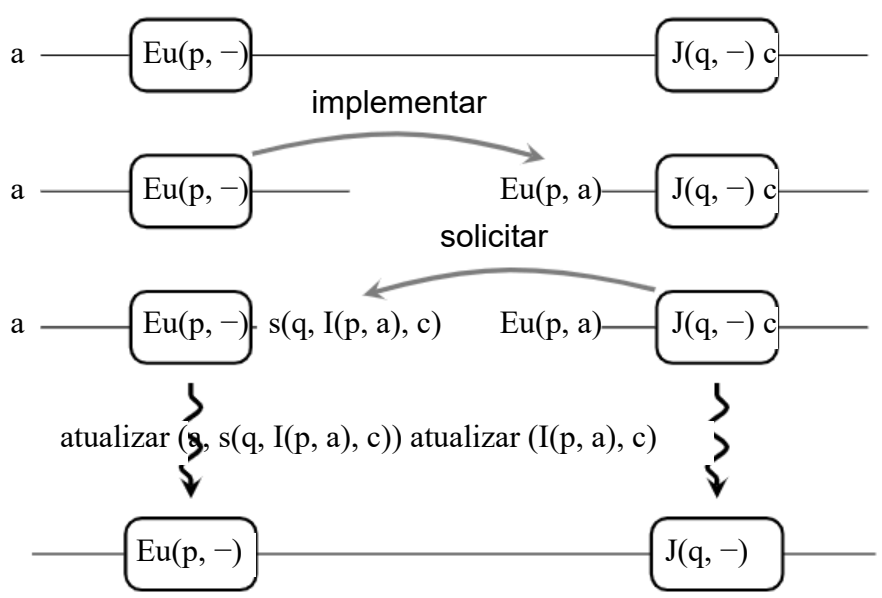


Figura 2. Uma função de solicitação permite que uma função de atualização seja definida para o composto $J(q, I(p, -))$.

Neste artigo, mostraremos que a aprendizagem se torna composicional — ou seja, podemos definir um algoritmo de aprendizagem $A \to C$ a partir dos algoritmos de aprendizagem $A \to B$ e $B \to C$ — desde que cada algoritmo de aprendizagem consista nestes quatro componentes:

- um espaço de parâmetros P,
- uma função de implementação $I : P \times A \to B$,
- uma função de atualização $U : P \times A \times B \to P$, e
- uma função de solicitação $r : P \times A \times B \to A$.

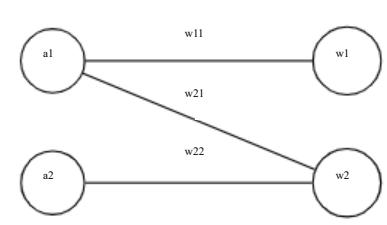
Mais precisamente, mostraremos que algoritmos de aprendizagem (P, I, U, r) formam os morfismos de uma categoria Aprender. Uma categoria é uma estrutura algébrica que modela composições ção. Mais precisamente, uma categoria consiste em tipos A, B, C e assim por diante, morfismos $f : A \to B$ entre esses tipos, e uma regra de composição pela qual os morfismos $f : A \to B$ e $g : B \to C$ podem ser combinados para criar um morfismo $A \to C$. Assim, podemos dizer que algoritmos de aprendizagem formam uma categoria, como explicamos informalmente acima. Na verdade, eles têm mais estrutura porque podem ser compostos não apenas em série, mas também em paralelo, e isso também tem uma descrição algébrica limpa. Ou seja, mostraremos que Learn tem a estrutura de uma categoria monoidal simétrica.

Esta nova categoria, Aprender, sintetizada a partir da análise acima de algoritmos de aprendizagem, curiosamente se assemelha e está intimamente relacionada a lentes [3] e jogos abertos [11], duas estruturas bem conhecidas que também modelam informações de troca composicional e bidirecional entre sistemas em interação. Retornaremos a isso brevemente na Seção VII.

Nosso objetivo até agora foi construir uma descrição algébrica de algoritmos de aprendizagem, e afirmamos que a categoria Aprender é suficiente. Em particular, então, nossa estrutura deve ser ampla o suficiente para abranger métodos conhecidos para a construção de algoritmos de aprendizagem supervisionada; tais algoritmos de aprendizagem devem estar dentro de Aprender como um tipo particular de morfismo. Aqui, estudamos redes neurais.

Digamos que uma camada de rede neural do tipo (n, n) é um subconjunto $C \subseteq [n] \times [n]$, onde $n, n \in N$ são números naturais e $[n] = \{1, \ldots, n\}$ para qualquer $n \in N$. Os números na e n representam o número de nós em cada lado da camada, C é o conjunto de conexões e a inclusão $C \subseteq [n] \times [n]$ codifica as informações de conectividade, ou seja, $(i, j) \in C$ significa que o nó i à direita está conectado ao nó j à esquerda.

Se adicionalmente fixarmos uma função $\sigma : R \to R$, que chamamos de função de ativação, então uma camada de rede neural define uma função parametrizada $I : R \times R \to R$. O Rfactor codifica números chamados pesos e o Rfactor codifica números chamados vieses. Por exemplo, a camada $C = \{(1, 1), (2, 1), (2, 2)\} \subseteq [2] \times [2]$, tem $n = 2$ vieses e $|C| = 3$ pesos. Os vieses são representados pelos nós à direita abaixo, enquanto os pesos são representados pelas arestas:



Esta camada define a função parametrizada $I : R \times R \to$

R, dado por

$$I\left(w, w, w, w, w, a, a\right) := \left(\sigma(wa + w), \sigma(wa + wa + w)\right).$$

Uma rede neural é uma sequência de camadas dos tipos (n, n), (n, n), . . . , (n, n). Ao compor as funções parametrizadas definidas por cada camada, como acima, a própria rede neural define uma função parametrizada $P \times R \to R$ para algum P. Observe que essa função é sempre diferenciável se σ for .

Para passar de uma função parametrizada diferenciável para um algoritmo de aprendizagem, normalmente especifica-se uma função de erro adequada e e um tamanho de passo ε, e então usa-se um algoritmo conhecido como gradiente descendente.

Nosso teorema principal é que, sob condições gerais, a descida do gradiente é composicional. Isso é formalizado como um functor Para → Learn, onde Para é uma categoria onde morfismos são funções parametrizadas diferenciáveis $I : P \times R \to R$ entre espaços euclidianos de dimensão finita, onde o espaço de parâmetros P = R também é euclidiano.

Em resumo, a funcionalidade significa que, dadas duas funções parametrizadas diferenciáveis I e J, obtemos o mesmo resultado se (i) usarmos a descida do gradiente para obter algoritmos de aprendizado para I e J e, em seguida, compormos esses algoritmos de aprendizado, ou (ii) compormos I e J como funções parametrizadas e, em seguida, usarmos a descida do gradiente para obter um algoritmo de aprendizado. Mais precisamente, temos o seguinte:

Teorema. Fixe $\varepsilon > 0$ e $e(x, y) : R \times R \to R$ tal que $\frac{\partial e}{\partial x}(x, -) : R \to R$ é invertível para cada $x \in R$. Então existe um functor monoidal simétrico, injetivo sobre objetos, fiel

$$L: \text{Para} \longrightarrow \text{Aprender}$$

enviando cada função parametrizada diferenciável $I : P \times R \to R$ para o algoritmo de aprendizagem (P, I, U, r) definido por

$$U(p, a, b) := p - \varepsilon \nabla E(p, a, b)$$

e

$$r(p, a, b) := f\left(\sum \nabla E(p, a, b)\right),$$

onde $E(p, a, b) = e(I(p, a), b)$ e f denota a aplicação componente a componente do inverso a (a, −) para cada i.

Este teorema tem uma série de consequências. Por enquanto, vamos citar apenas três. A primeira é que podemos treinar uma rede neural usando os dados de treinamento de toda a rede para criar dados de treinamento para cada subunidade e, em seguida, treinar cada subunidade separadamente. Até certo ponto, isso é bem conhecido: é responsável por acelerações devido à retropropagação, já que nunca é necessário calcular as derivadas da função definida por toda a rede.

Entretanto, o fato de esse functor ser simétrico monoidal mostra que podemos variar o algoritmo de retropropagação para fatorar a rede neural em subpartes mais ricas do que simplesmente dividi-la camada por camada.

Em segundo lugar, ele fornece uma condição suficiente — que é direta e geral — sob a qual uma função de erro funciona bem na retropropagação.

Por fim, demonstra que a retropropagação pode ser aplicada de forma muito mais geral do que apenas a redes neurais: ela é composicional para todas as funções parametrizadas diferenciáveis. Consequentemente, demonstra que a retropropagação fornece um método sólido para calcular a descida do gradiente, mesmo se introduzirmos elementos muito mais gerais em redes neurais do que as composições tradicionais de funções lineares e funções de ativação.

Visão geral

Na Seção II, definimos a categoria Aprendizagem de algoritmos de aprendizagem. Apresentamos o teorema principal na Seção III: dada uma escolha de função de erro e tamanho de passo, a descida do gradiente e a retropropagação fornecem um functor da categoria de funções parametrizadas para a categoria de algoritmos de aprendizagem. Na Seção IV, ampliamos essa visão para mostrar como ela se relaciona com redes neurais. Em seguida, na Seção V, observamos que a categoria Aprendizagem tem uma estrutura adicional além daquela de uma categoria monoidal simétrica: ela tem estruturas bimonoides que nos permitem dividir e mesclar conexões para formar redes. Também mostramos que isso é útil para entender a construção de neurônios individuais e para vinculação de pesos e redes neurais convolucionais. Em seguida, calculamos explicitamente um exemplo de funcionalidade de redes neurais para algoritmos de aprendizagem (§VI) e discutimos implicações para essa estrutura (§VII). A versão estendida [10] deste artigo fornece apêndices com aspectos mais técnicos da prova do teorema principal e uma breve introdução, baseada em diagramas, a tópicos relevantes na teoria das categorias.

II. A categoria dos alunos

Nesta seção, definimos uma categoria monoidal simétrica, Learn, que modela algoritmos de aprendizado supervisionado e seus compostos. Consulte a versão estendida [10] para obter informações básicas sobre categorias e diagramas de strings.

Definição II.1. Sejam A e B conjuntos. Um algoritmo de aprendizado supervisionado, ou simplesmente aprendiz, A → B é uma tupla (P, I, U, r) onde P é um conjunto, e I, U e r são funções dos tipos:

$$Eu : P \times A \to B,$$

$$U : P \times A \times B \to P,$$

$$r : P \times A \times B \to A.$$

Chamamos P de espaço de parâmetros; ele é apenas um conjunto. O mapa I implementa um valor de parâmetro $p \in P$ como uma função

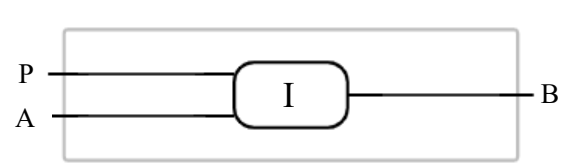
$I(p, -) : A \to B$. Pensamos em um par $(a, b) \in A \times B$ como um dado de treinamento; ele pareia uma entrada a com uma saída b. O mapa $U : P \times A \times B \to P$ é a função de atualização; dado um parâmetro 'atual' p e um dado de treinamento $(a, b) \in A \times B$, ele produz um parâmetro 'atualizado' $U(p, a, b) \in P$. Isso pode ser considerado como a etapa de aprendizado. A ideia é que a função atualizada $I(U(p, a, b), -) : A \to B$ esperançosamente enviaria a mais perto de b do que a função $I(p, -)$ fez, embora isso não seja um requisito e certamente nem sempre é verdade na prática. Por fim, temos a função de solicitação $r : P \times A \times B \to A$. Ela usa o mesmo dado e produz um 'valor solicitado' $r(p, a, b) \in A$. A ideia é que esse valor seja enviado aos alunos a montante para seu próprio treinamento.

Observação II.2. A função de solicitação talvez seja um pouco misteriosa neste estágio. De fato, ela é supérflua para a definição de um algoritmo de aprendizado independente: tudo o que precisamos para aprender é um espaço P de funções $I(p, -)$ para pesquisar e uma regra U para atualizar nosso parâmetro p à luz de novas informações. Como enfatizamos na introdução, a função de solicitação é crucial na composição de algoritmos de aprendizado: não há regra de atualização composta sem a função de solicitação.

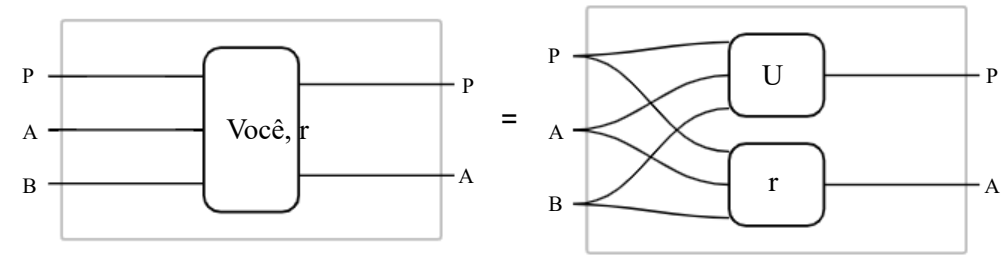
Outra maneira de entender o papel da função de solicitação vem de experimentos em aprendizado de máquina. Fixando algum parâmetro p e, portanto, uma função $I(p, -)$, a função de solicitação nos permite escolher uma saída desejada b e, então, para qualquer entrada a, retornar uma nova entrada $a := r(p, a, b)$. No caso da retropropagação, veremos que temos a intuição de que $I(p, a)$ está mais próximo de b do que $I(p, a)$. Por exemplo, se estivermos classificando imagens e b for o valor que indica a classificação "gato", então a será uma versão mais "semelhante a um gato" da imagem a. Isso é semelhante em espírito ao que tem sido denominado inversão ou "sonho" em redes neurais [17].

Uma terceira maneira de entender a função de solicitação é por analogia com outras estruturas composicionais: a função de solicitação desempenha um papel análogo à função de colocação em uma lente assimétrica [3] e à função de coplay em um jogo aberto [11].

Observação II.3. Usando diagramas de string em (Conjunto, ×), podemos desenhar uma função de implementação I da seguinte forma:



Pode-se fazer o mesmo para U e r, embora consideremos conveniente combiná-los em uma única solicitação de atualização função $(U, r) : P \times A \times B \to P \times A$. Esta função pode ser desenhada da seguinte forma:



Sejam (P, I, U, r) e (P, I, U, r) aprendizes do tipo $A \to B$. Consideramos que eles são equivalentes se houver uma bijeção $f : P \to P$ tal que o seguinte seja válido para cada $p \in P$, $a \in A$ e $b \in B$:

$$Eu(f(p), a) = I(p, a),$$
$$U(f(p), a, b) = f(U(p, a, b)),$$
$$r(f(p), a, b) = r(p, a, b).$$

De fato, uma noção mais forte de equivalência — a relação de equivalência gerada pela existência de uma sobrejeção f com essas propriedades — também faz sentido semântico, mas usamos essa definição porque ela dá origem à fidelidade no teorema principal (Teorema III.2).

Proposição II.4. Existe uma categoria monoidal simétrica Learn cujos objetos são conjuntos e cujos morfismos são classes de equivalência de aprendizes.

A demonstração da Proposição II.4 é apresentada no Apêndice A da versão estendida [10]. Por enquanto, simplesmente especificamos a composição, as identidades, o produto monoidal e a trança para esta categoria monoidal simétrica. Observe que, embora escrevamos em termos de representantes, cada um deles é bem definido, respeitando a relação de equivalência em aprendizes.

a) Composição: Suponha que temos um par de alunos $A \xrightarrow{(P,I,U,r)} B \xrightarrow{(Q,J,V,s)} C$.

O aluno composto $A \to C$ é definido como $(P \times Q, I * J, U * V, r * s)$, onde a função de implementação é

$$(I * J)(p, q, a) := J(q, I(p, a))$$

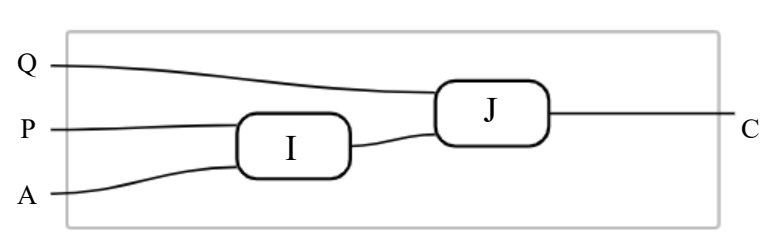
a função de atualização é

$$(U * V)(p, q, a, c) := \left(U\left(p, a, s(q, I(p, a), c)\right), V\left(q, I(p, a), c\right)\right),$$

e a função de solicitação

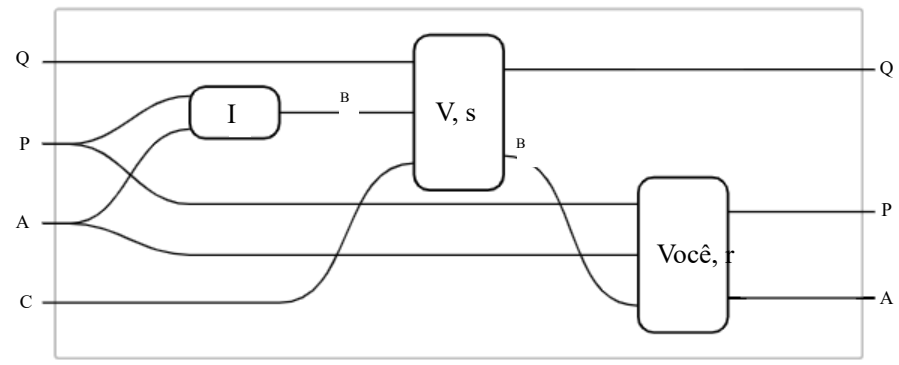
$$(r * s)(p, q, a, c) := r\left(p, a, s(q, I(p, a), c)\right).$$

Vamos também apresentar a regra de composição usando diagramas de strings em (Conjunto, ×). Dadas as variáveis (P, I, U, r) e (Q, J, V, s) como acima, a função de implementação composta pode ser escrita como



[1] Diagramas de strings são uma sintaxe alternativa, mas ainda formal, para morfismos em uma categoria monoidal. Veja a versão estendida [10] para mais detalhes.

enquanto a função composta de atualização-solicitação (U ∗ V, r ∗ s) pode ser escrita como:



Aqui a divisão representa o mapa diagonal A → A × A, ou seja, a 7 → (a, a).

Esperamos que o leitor ache o traçado visual destes diagramas útil para compreender a regra de composição. Para repetir a intuição da introdução, suponha que, dados os parâmetros atuais p ∈ P e q ∈ Q, e os dados de treinamento a ∈ A e c ∈ C. I toma p e a e produz algum b ∈ B para treinar o segundo componente. Juntamente com q e c, b é usado para calcular um parâmetro atualizado q juntamente com um valor b para treinar o primeiro componente. Juntamente com p e a, b é usado para calcular um parâmetro atualizado p juntamente com um valor a.

b) Identidades: Para cada objeto A, temos o mapa de identidade

$$(R, id, !, \pi) : A \longrightarrow A,$$

onde id : R× A → A é a segunda projeção (como esta é uma bijeção, abusamos da notação para escrever esta projeção como id), ! : R× A × A → R é a função única, e π: R× A × A → A é a projeção no fator final (novamente, ignorando o R).

c) Produto monoidal: O produto monoidal dos objetos A e B é simplesmente seu produto cartesiano A × B como conjuntos. O produto monoidal dos morfismos (P, I, U, r): A → B e (Q, J, V, s): C → D é definido como (P × Q, I ∥ J, U ∥ V, r ∥ s), onde a função de implementação é

$$(I \parallel J)(p, q, a, c) \coloneqq (I(p, a), J(q, c))$$

a função de atualização é

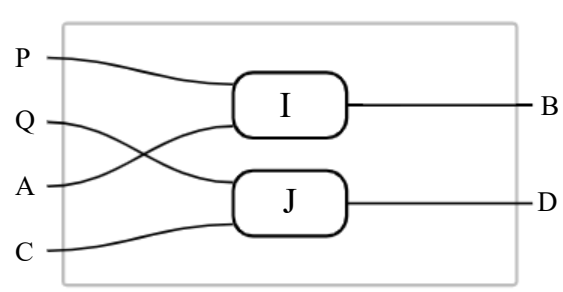
$$(U \parallel V)(p, q, a, c, b, d) \coloneqq (U(p, a, b), V(q, c, d))$$

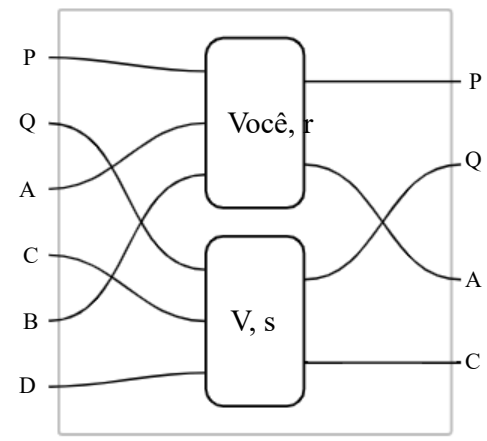
e a função de solicitação é

$$(r \parallel s)(p, q, a, c, b, d) \coloneqq (r(p, a, b), s(q, c, d)).$$

Usamos a notação ∥ porque o produto monoidal pode ser considerado uma composição paralela — e não em série.

Também apresentamos isso em diagramas de strings:





d) Trança: Uma trança simétrica A × B → B × A é dada por (R, σ, !, σ∘π) onde σ : A×B → B×A é a função de troca usual (a, b) 7 → (b, a) e π : R×(A×B)×(B×A) → B × A é novamente a projeção no fator final.

Uma prova de que esta é uma categoria monoidal simétrica bem definida pode ser encontrada na versão estendida [10].

III. Descida do gradiente e retropropagação

Nesta seção, mostramos que a descida do gradiente e a retropropagação definem um functor monoidal simétrico forte de uma categoria monoidal simétrica Para, de funções parametrizadas diferenciáveis entre espaços euclidianos de dimensão finita, para a categoria monoidal simétrica Learn de algoritmos de aprendizagem.

Primeiro definimos a categoria de funções parametrizadas diferenciáveis. Um espaço euclidiano é um da forma R para algum n ∈ N. Chamamos n de dimensão do espaço e escrevemos um elemento a ∈ Ras (a, . . . , a), ou simplesmente (a), onde cada a∈ R. . Para espaços euclidianos A = R B = R, defina uma função parametrizada diferenciável A → B como um par (P, I), onde P é um espaço euclidiano e I : P × A → B é uma função diferenciável. Chamamos dois desses pares de (P, I), (P, I) equivalentes se existe uma bijeção diferenciável f : P → P tal que para todo p ∈ P e a ∈ A temos I(f (p), a) = I(p, a). Funções parametrizadas diferenciáveis entre espaços euclidianos formam uma categoria monoidal simétrica.

Definição III.1. Escrevemos Para para a categoria monoidal simétrica estrita cujos objetos são espaços euclidianos e cujos morfismos R→ Classes de equivalência raras de funções parametrizadas diferenciáveis R→ R.

Composição de (P, I): R→ Rand (Q, J): R→ R é dada por (P × Q, I ∗ J) onde

$$(I * J)(p, q, a) = J(q, I(p, a)).$$

O produto monoidal dos objetos Rand R é o objeto R, enquanto o produto monoidal dos morfismos (P, I) : R→ R e (Q, J) : R→ R é dado por (P × Q, I ∥ J) onde

$$(I \parallel J)(p, q, a, c) = \left(\quad \right) Eu(p, a), J(q, c).$$

A trança R∥ R→ R∥ R é dada por (R, σ) onde σ(a, b) = (b, a).

É fácil verificar se esta é uma categoria monoidal simétrica bem definida. Agora estamos em condições de enunciar o teorema principal.

Teorema III.2. Fixe um número real $\varepsilon > 0$ e $e(x, y) : R \times R \to R$ diferenciável tal que $(x, -) : R \to R$ é invertível para cada $x \in R$. Então podemos definir um functor monoidal simétrico forte, injetivo-onobjects e fiel

$$L: \text{Para} \to \text{Aprender}$$

que envia cada função parametrizada $I : P \times A \to B$ para o aluno (P, I, U, r) definida por

$$U(p, a, b) := p - \varepsilon \nabla E(p, a, b)$$

e

$$r(p, a, b) := f\left(\nabla E(p, a, b)\right),$$

onde $E(p, a, b) := \sum_j e(I(p, a)_j, b_j)$ e $f(b) := a$ é a aplicação componente a componente do inverso a $(a, -)$ para cada i.

Demonstração (esboço). A demonstração deste teorema consiste em observar que a regra da cadeia é funcional, dada a configuração acima. Os pontos-chave são o uso da regra da cadeia para demonstrar a funcionalidade da parte P da função de atualização e da função de solicitação. Uma demonstração completa é apresentada na versão estendida [10].

Chamamos ε de tamanho do passo, e de função de erro e E de erro total (em relação a I). Também chamamos os functores L, assim chamados porque transformam funções parametrizadas em algoritmos de aprendizagem, de functores de descida de gradiente/retropropagação.

Observação III.3. A função de atualização U codifica o que é conhecido como descida do gradiente: o parâmetro p é atualizado movendo-o um passo ε na direção que mais reduz o erro total E.

A função de solicitação recodifica o valor de retropropagação, retornando o gradiente do erro total em relação à entrada a, conforme modificado pela função invertível f. Para dar um exemplo, a funcionalidade de L diz que as duas funções de atualização a seguir são iguais:

- A função de atualização U, que representa a descida do gradiente no composto de funções parametrizadas $((I \parallel J) * K) * M$.
- A função de atualização $((U \parallel U) * U) * U$, que representa o composto, de acordo com a estrutura em Learn, das funções de atualização U, U, U e U juntamente com as funções de solicitação r, r, r e r.

Isso mostra que podemos calcular a descida do gradiente por meio do cálculo local do gradiente, juntamente com a passagem de mensagens local. Este é o algoritmo de retropropagação.

Exemplo III.4 (Erro quadrático). O erro quadrático é dado pela função de erro $e(x, y) := (x - y)$, de modo que o erro total é dado por

$$E(p, a, b) = \sum_j (I(p, a) - b) = \|I(p, a) - b\|.$$

Neste caso, $(x, -)$ é a função $y \mapsto x - y$. Esta função é sua própria inversa, então temos $f(y) := x - y$.

Fixando algum tamanho de passo $\varepsilon > 0$, temos

$$\begin{aligned} U(p, a, b) &:= p - \varepsilon \nabla E(p, a, b) \\ &= \left(p - \varepsilon \sum_j (I(p, a) - b)\,(p, a)_k\right) \end{aligned}$$

e da mesma forma

$$\begin{aligned} r(p, a, b) &:= a - \nabla E(p, a, b) \\ &= \left(\text{uma} - \sum_j (I(p, a) - b)\,(p, a)_i\right). \end{aligned}$$

Assim, dada esta escolha de função de erro, o functor L do Teorema III.2 apenas implementa, como função de atualização, a descida de gradiente usual com tamanho de passo ε em relação ao erro quadrático.

Observação III.5. Comparando as solicitações r com as atualizações U no Exemplo III.4, pode-se notar que elas são semelhantes, exceto que a primeira parece não ter o ε. Alguém poderia se perguntar por que as duas são diferentes ou para onde foi o fator ε.

O Teorema III.2 mostra, no entanto, que, na verdade, a similaridade entre rand U é uma coincidência. O importante sobre as solicitações e, portanto, as mensagens repassadas na retropropagação, é o fato de que elas são construídas invertendo certas derivadas parciais que são então aplicadas ao gradiente do erro total em relação à entrada. Interpretamos o resultado como um valor de entrada "corrigido" que, se usado em vez disso, reduziria o erro total em relação à saída fornecida e ao valor do parâmetro atual. Em particular, a semelhança dos valores da solicitação com a descida do gradiente no Exemplo III.4 é apenas um artefato da escolha da função de erro quadrática $e(x, y) := (x - y)$.

IV. Das redes às funções parametrizadas

Na seção anterior, mostramos que a descida do gradiente e a retropropagação — para uma dada escolha de função de erro e tamanho de passo — definem um functor de funções parametrizadas diferenciáveis para algoritmos de aprendizado supervisionado. Mas a retropropagação é frequentemente considerada um algoritmo executado em uma rede neural. Como as redes neurais entram em cena? Como veremos, as redes neurais são um método para definir funções parametrizadas a partir de arquiteturas de rede.

Este método, assim como a própria retropropagação, também é composicional — ele respeita a colagem de redes neurais. Para formalizar isso, primeiro definimos uma categoria NNet de redes neurais. A implementação de cada rede neural definirá uma função parametrizada e, de fato, obtemos um functor I : NNet → Para. Observe que, assim como a definição de um functor de descida de gradiente/retropropagação depende de uma escolha (ou seja, da função de erro e do tamanho do passo), a definição de I também depende. Ou seja, devemos escolher uma função de ativação.

Lembre-se da introdução que uma camada de rede neural do tipo (m, n) é um subconjunto de [m]×[n], onde m, n ∈ N e [n] = {1, . . . , n}. Uma rede neural de k camadas do tipo (m, n) é uma sequência de camadas de rede neural dos tipos (n, n), (n, n), . . . , (n, n), onde n= m e n= n. Uma rede neural do tipo (m, n) é uma rede neural de k camadas para algum k.

Dada uma rede neural do tipo (m, n) e uma rede neural do tipo (n, p), podemos concatená-las para obter uma rede neural do tipo (m, p). Observe que, quando m = n, consideramos a rede neural da camada 0 como um morfismo. Concatenar qualquer rede neural em qualquer um dos lados com a rede neural da camada 0 não a altera.

Definição IV.1. A categoria NNet de redes neurais tem como objetos números naturais e como morfismos redes neurais m → n do tipo (m, n). A composição é dada pela concatenação de redes neurais. O morfismo identidade em n é a rede neural da camada 0.

Como a composição é apenas concatenação, ela é imediatamente associativa, e de fato definimos uma categoria.

Proposição IV.2. Dada uma função diferenciável σ : R → R, temos um functor

$$I: \mathrm{NNet} \longrightarrow \mathrm{Para}.$$

Em objetos, Imapeia cada número natural n para o espaço euclidiano ndimensional R.

Em morfismos, cada rede neural de 1 camada C : m → n é mapeada para a função parametrizada

$$\begin{aligned} Eu: R\times R &\longrightarrow R; \\ \big((w, w), x\big)_{\substack{1\le i\le m \\ 1\le j\le n}} &\mapsto \Big(\sigma\big(\textstyle\sum_i wx + w\big)\Big)_{1\le j\le n}. \end{aligned}$$

Dada uma rede neural N = C, . . . , C, a imagem sob I é o composto da imagem de cada camada:

$$Eu= Eu* \cdots * Eu$$

Chamamos σ de função de ativação, os wweights, onde (i, j) ∈ C, e os wbiases, onde j ∈ [n].

Prova. A prova desta proposição é simples

Observe, em particular, que a imagem I de cada camada C é diferenciável e, portanto, suas composições também o são. Observe também que a imagem de uma rede neural de camada 0 é a composição vazia, portanto, identidades são mapeadas para identidades. A composição é preservada por definição.

Compondo um functor de implementação I com um functor de descida de gradiente/retropropagação L, obtemos um functor

$$\begin{tikzcd} \mathrm{NNet} \arrow[rr] \arrow[dr, "I"'] & & \mathrm{Aprender} \\ & \mathrm{Pára} \arrow[ur, "L"'] & \end{tikzcd}$$

Isso afirma que, dadas as escolhas de função de ativação σ, função de erro e e tamanho do passo ε, uma rede neural define um algoritmo de aprendizado supervisionado e o faz de forma composicional.

Uma estrutura monoidal simétrica, tanto na NNet quanto nos functores acima, pode ser dada generalizando a categoria NNet para a categoria onde os morfismos são grafos acíclicos direcionados com interfaces; detalhes sobre tal categoria podem ser encontrados em [7], veja também Observação V.5.

Na Seção VI, computaremos um exemplo estendido do uso de redes neurais para definir composicionalmente algoritmos de aprendizado supervisionado. Antes disso, porém, discutiremos estruturas composicionais adicionais disponíveis para nós em Learn, Para e a generalização monoidal da NNet mencionada anteriormente.

V. Networking no Aprendizado

Nossa formulação de algoritmos de aprendizado supervisionado como morfismos em uma categoria monoidal significa que algoritmos de aprendizado podem ser formados pela combinação de outros algoritmos de aprendizado em sequência e em paralelo. De fato, como sugerido pelas próprias redes neurais, temos mais estrutura disponível: somos capazes de formar novos algoritmos de aprendizado combinando outros em redes de aprendizes onde os fios podem se dividir e se fundir. Formalmente, isso significa que cada objeto na categoria de aprendizes é equipado com a estrutura de um bimonoide.

Para isso, observe primeiro que a categoria monoidal simétrica FVect de aplicações lineares entre espaços euclidianos está dentro da categoria Para de funções parametrizadas; simplesmente consideramos cada aplicação linear como parametrizada pelo espaço de parâmetros trivial R. Dada uma escolha de tamanho de passo e função de erro, e portanto um functor L: Para → Learn como no Teorema III.2, temos uma inclusão

$$\mathrm{FVect} \longrightarrow \mathrm{Para} \xrightarrow{L} \mathrm{Aprenda}.$$

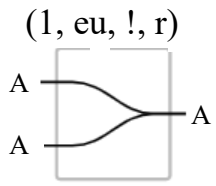
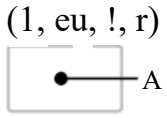
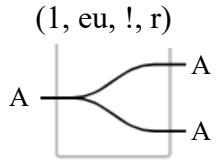
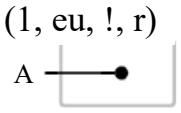
Isso nos permite construir um algoritmo de aprendizagem — isto é, um morfismo em Learn — como a imagem de qualquer morfismo em FVect, e os algoritmos de saída obedecem às mesmas equações dos mapas lineares de entrada. Em particular, a partir da álgebra linear gráfica [2], [4], sabemos que cada objeto em FVect é equipado com uma estrutura bimonoide, então

Podemos usar nosso functor L para equipar cada objeto da forma Rin Learn com uma estrutura bimonoide. Essa estrutura bimonoide é o que torna a notação de rede neural viável: podemos interpretar a divisão e a combinação de forma coerente com a composição.

De fato, os bimonoides construídos dependem apenas da escolha da função de erro; não precisamos especificar o tamanho do passo. Como exemplo, detalhamos a construção usando retropropagação em relação ao erro quadrático (Exemplo III.4).

Proposição V.1. O gradiente descendente em relação ao erro quadrático e ao tamanho do passo ε define um functor monoidal simétrico FVect → Learn. Isso implica que cada objeto na imagem desse functor pode ser equipado com a estrutura de um bimonoide.

Explicitamente, os mapas bimonoides são fornecidos da seguinte forma. Observe que todos eles têm um espaço de parâmetros trivial R, então denotamos a função de atualização única ! : R× A × B → R.

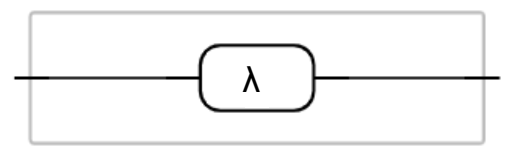
	Solicitação de Implementação	
Multiplicação μ (1, eu, !, r) 	Eu(a, a) = a+ a	r((a, a), a) = (a− a, a− a)
Unidade η (1, eu, !, r) 	Eu(0) = 0 r(a) = 0	
Comultiplicação δ (1, eu, !, r) 	Eu(a) = (a, a)	r(a, (a, a)) = a+ a− a
Condado ϱ (1, eu, !, r) 	Eu(a) = 0 r(a) = 0	

Observação V.2. Na verdade, temos muitas estruturas bimonoides diferentes em Learn: cada escolha de função de erro define uma, e estas são frequentemente distintas. Por exemplo, se escolhermos e(x, y) = xy, então a função de solicitação na multiplicação é dada por r(a, a, a) = (a, a) e a função de solicitação na comultiplicação é dada por r(a, a, a) = a+ a. Embora esta seja uma função de erro bastante estranha — minimizar o erro implica enviar saídas para 0 — a existência de tais estruturas é interessante.

Uma escolha de estruturas bimonoides, como a apresentada pela Proposição V.1, permite-nos interpretar diagramas de rede na categoria monoidal (Aprenda, ‖) da Proposição II.4. De fato, eles fornecem interpretações canônicas de divisão, junção, inicialização e descarte de fios.

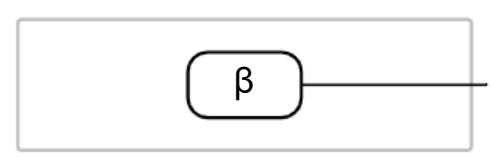
Exemplo V.3 (Construindo neurônios). Como algoritmos de aprendizagem implementados com relação ao erro quadrático (ver Exemplo III.4) e a um certo tamanho de passo ε, as redes neurais têm uma estrutura bastante simples: são geradas por três algoritmos básicos de aprendizagem — multiplicação escalar λ, viés β e uma função de ativação σ — juntamente com a multiplicação bimonoide μ e a comultiplicação δ dadas pela Proposição V.1.

O algoritmo de aprendizagem de multiplicação escalar λ : R → R, que representaremos graficamente pelo diagrama de strings em Learn



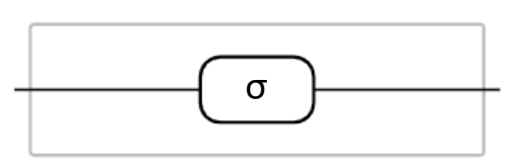
é dado pelo espaço de parâmetros R, função de implementação λ(w, x) = wx, função de atualização U(w, x, y) = w − εx(wx− y) e função de solicitação r(w, x, y) = x−w(wx− y).

O algoritmo de aprendizagem de viés β : R→ R, que representamos



é dado pelo espaço de parâmetros R, implementação β(w) = w, função de atualização U(w, y) = (1 − ε)w + εy e função de solicitação trivial, uma vez que tem espaço de entrada trivial.

O algoritmo de aprendizagem da função de ativação σ : R → R, representado

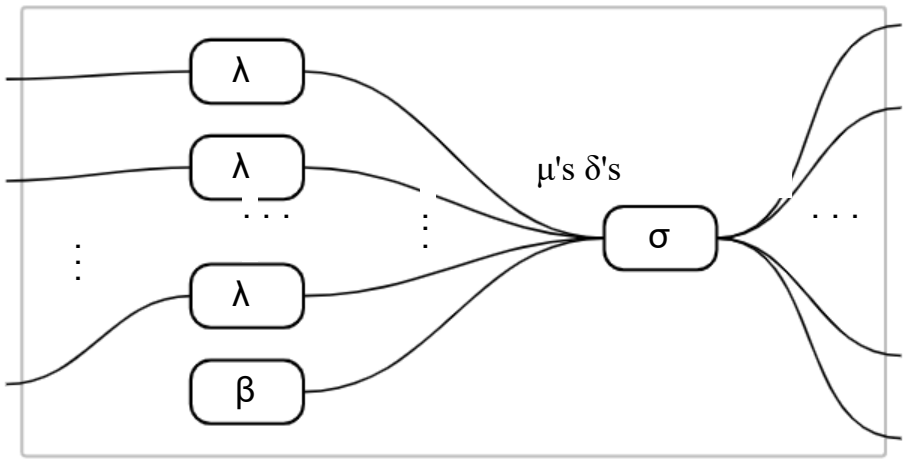


tem espaço de parâmetros trivial e é especificado por alguma escolha de função de ativação σ : R → R, juntamente com a função de atualização trivial e a função de solicitação r(x, y) = x − (x − y) (x).

Então, cada neurônio em uma rede neural pode ser entendido como um composto desses geradores da seguinte forma: primeiro, um produto monoidal do número necessário de algoritmos de multiplicação escalar e um algoritmo de polarização, depois um composto de μ's, uma função de ativação e, finalmente, um

[2] Observe que estes são diagramas de strings em (Aprender, ‖), enquanto os diagramas de strings da Seção II eram diagramas de strings em (Conjunto, ×). Como sempre, os diagramas de strings representam morfismos em uma categoria, com o domínio à esquerda do diagrama e o contradomínio à direita. Para mais detalhes, consulte a versão estendida [10].

composto de δs.

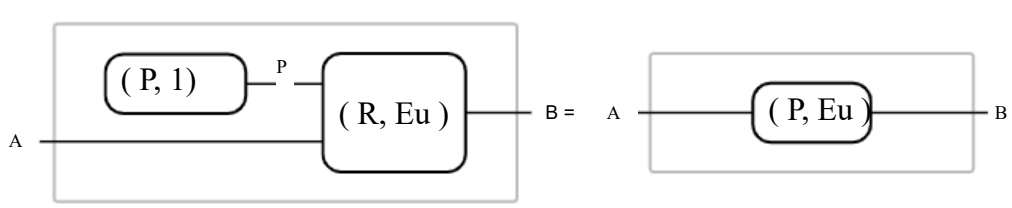


A composição dessas unidades usando a regra de composição em Learn constrói ainda mais qualquer algoritmo de aprendizado que pode ser obtido por descida de gradiente e retropropagação em uma rede neural com relação ao erro quadrático.

Exemplo V.4 (Vinculação de Pesos). A vinculação de pesos (ou compartilhamento de pesos) em redes neurais é um método pelo qual parâmetros em diferentes partes da rede são restringidos a serem iguais. É usado em redes neurais convolucionais, por exemplo, para forçar a rede a aprender os mesmos tipos de formas básicas que aparecem em diferentes partes de uma imagem [13]. Isso é facilmente representado em nossa estrutura. Antes de explicar como isso funciona, primeiro explicamos uma maneira de fatorar morfismos em Para em partes básicas.

Morfismos em Para são gerados, grosso modo, por morfismos de dois tipos diferentes: funções trivialmente parametrizadas e constantes parametrizadas. Dada uma função diferenciável $f : R \to R$, consideramos uma função trivialmente parametrizada $R \times R \to R$, cujo espaço de parâmetros $P = R$ é um ponto. Por constante parametrizada, entendemos um morfismo identidade $1: P \to P$, considerado como uma função parametrizada $P \times R \to P$.

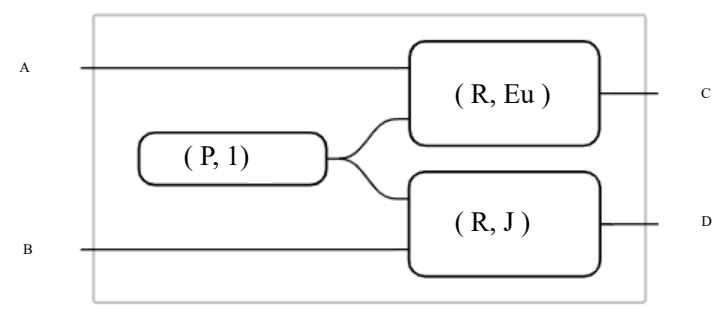
Em particular, toda função parametrizada pode ser escrita como um composto, usando a estrutura bimonoide, de uma função trivialmente parametrizada e uma constante parametrizada. Para ver isso, usamos diagramas de strings em (Para, ‖), onde aqui denotamos uma função parametrizada $I : P \times A \to B$ como uma caixa rotulada (P, I) com entrada A e saída B. É fácil verificar que qualquer função parametrizada $I : P \times A \to B$ é o composto de uma função trivialmente parametrizada e uma constante parametrizada, como segue:



Como esses morfismos são os mesmos em Para, eles correspondem ao mesmo algoritmo de aprendizado, pela funcionalidade de L, Teorema III.2. Observando a figura à direita, suponha que seja dado um dado de treinamento (a, b). O bloco (R, I) possui um espaço de parâmetros trivial, portanto, atualizações nele não fazem nada; no entanto, ele é capaz de enviar uma solicitação à entrada A e ao bloco (P, 1). O bloco (P

1) O bloco então executa a atualização desejada. Novamente, o resultado deve ser o mesmo, pelo teorema principal.

Isso sugere como devemos pensar na amarração de pesos. A ideia esquemática, representada em diagramas de strings, é a seguinte:



A estrutura comonoide da Proposição V.1 nos diz como a rede acima se comportará como um algoritmo de aprendizado em relação ao erro quadrático. O fio de divisão enviará o mesmo parâmetro para ambas as implementações I e J e se atualizará com base na soma das solicitações recebidas de I e J.

Observação V.5. Foi adequado aos nossos propósitos simplesmente considerar a categoria NNet das redes neurais. Dito isso, as redes neurais intuitivamente possuem estruturas monoidal e bimonoide: podemos colocar redes lado a lado para representar duas redes executadas em paralelo, e podemos adicionar múltiplas entradas e saídas duplicadas a cada nó de uma rede neural, conforme desejarmos.

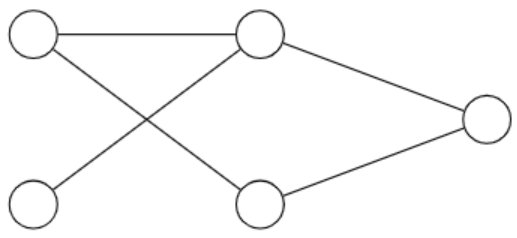
De fato, a categoria NNet pode ser generalizada para uma categoria monoidal simétrica com bimonoides em cada objeto. Essa generalização é a categoria monoidal simétrica estrita IDAG de idags — grafos acíclicos direcionados interligados — que foi estudada anteriormente como uma estrutura importante em concorrência, bem como por suas elegantes propriedades categóricas [7].

Também é desejável que cada functor I: NNet → Para, implementando redes neurais como funções parametrizadas, fatorize como NNet → IDAG → Para, e isso de fato pode ser feito. Além disso, o fator IDAG → Para é simétrico monoidal e preserva estruturas bimonoides.

VI. Exemplo: aprendizagem profunda

Nesta seção, calculamos explicitamente um exemplo da funcionalidade da implementação de uma rede neural como um algoritmo de aprendizado supervisionado. Para isso, fixamos uma função de ativação σ, bem como a função de erro quadrático, e um tamanho de passo $\varepsilon > 0$. Isso define, respectivamente, os functores I: NNet → Para e L: Para → Learn pelo Teorema III.2 e pela Proposição IV.2. Em particular, veremos que L implementa o algoritmo de retropropagação usual com erro quadrático e tamanho de passo ε em uma rede neural com função de ativação σ. Para simplificar a notação, escreveremos I para I.

Considere a seguinte rede, que tem uma única camada oculta:



Chame esta rede de A; ela é um morfismo A: 2 → 1 na categoria NNet das redes neurais. A imagem de A sob o functor I: NNet → Para é a função parametrizada I: (R × R) × R−→ R definida por

$$Eu(p, q, a) = \sigma\begin{pmatrix} q\sigma(pa+ pa+ p) + q\sigma(pa+ p) + q \end{pmatrix}.$$

Aqui, o espaço de parâmetros é R × R, pois há um peso para cada uma das três arestas na primeira camada, um viés para cada um dos dois nós na coluna intermediária, um peso para cada uma das duas arestas na segunda e um viés para o nó de saída. O espaço de entrada é R, pois há dois neurônios no lado mais à esquerda da rede, e o espaço de saída é R, pois há um único neurônio no lado mais à direita.

Escrevemos as entradas do espaço de parâmetros R× Ras p, p, p, p, p, q, q e q, onde prerepresenta o peso na aresta do i-ésimo nó da primeira coluna até o j-ésimo nó da segunda coluna, prerepresenta o viés no j-ésimo nó da segunda coluna, qrepresenta o peso na aresta do j-ésimo nó da segunda coluna até o nó único da coluna final e q representa o viés no nó de saída.

Suponha que desejamos treinar esta rede. Um método de treinamento é fornecido pelo functor L, que transforma esta função parametrizada I em um algoritmo de aprendizado supervisionado. Em particular, dado um par de dados de treinamento (a, c) em R × R, desejamos obter um mapa R × R → R × R que atualize o valor de (p, q). Como optamos por definir L usando a descida do gradiente em relação à função de erro quadrática e um tamanho de passo ε, este mapa é precisamente o mapa de atualização fornecido pela L-imagem de I em Learn. Ou seja, esta função parametrizada mapeia para o algoritmo de aprendizado (R × R, I, U, r), onde

$$U: (R\times R) \times R\times R \longrightarrow R\times R$$

é definido por

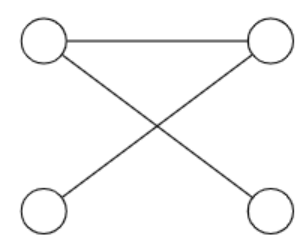
$$U(p, q, a, c) = \begin{pmatrix} p - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)q\,\dot\sigma(\beta)ap - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)q\,\dot\sigma(\beta)ap - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)q\,\dot\sigma(\beta)ap - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)q\,\dot\sigma(\beta)\,p - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)q\,\dot\sigma(\beta)\,q - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)\sigma(\beta) \\ q - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)\sigma(\beta) \\ q - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma) \end{pmatrix},$$

e r: (R× R) × R× R −→ R é definido por

$$r(p, q,a, c) = a - \nabla\|I(p, q, a) - c\| = \begin{pmatrix} uma - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)(q\,\dot\sigma(\beta)p + q\,\dot\sigma(\beta)p) \\ uma - \varepsilon(I(p, q, a) - c)\,\dot\sigma(\gamma)q\,\dot\sigma(\beta)p \end{pmatrix}$$

onde γ é tal que I(p, q, a) = σ(γ), onde β= pa+ pa+ p, onde β= pa+ p, e onde ˙σ é a derivada da função de ativação σ. (Explicitamente, γ = qσ(pa+ pa+ p) + qσ(pa+ p) + q.) Observe que U executa a descida do gradiente conforme reivindicado.

A expressão acima para U é complexa. No entanto, ela reutiliza cálculos como γ, β e β repetidamente. Para simplificar o cálculo, podemos tentar fatorá-la. Uma fatoração pode ser obtida a partir da própria rede neural. Observe que a rede acima pode ser escrita como a composição de duas camadas. A primeira camada B : 2 → 2



mapas para a função parametrizada

$$Eu: R\times R \longrightarrow R; \quad (p, a) \mapsto \begin{pmatrix} \sigma(pa+ pa+ p) \\ \sigma(pa+ p) \end{pmatrix}$$

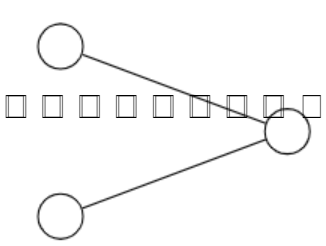
que por sua vez possui funções de atualização e solicitação

$$U: R\times R\times R \longrightarrow R; \quad (p, a, b) \mapsto \begin{pmatrix} p - \varepsilon(I(p, a) - b)\,\dot\sigma(\beta)ap - \varepsilon(I(p, a) - b)\,\dot\sigma(\beta)ap - \varepsilon(I(p, a) - b)\,\dot\sigma(\beta)ap - \varepsilon(I(p, a) - b)\,\dot\sigma(\beta)\,p - \varepsilon(I(p, a) - b)\,\dot\sigma(\beta) \end{pmatrix}$$

e r: R× R× R−→ R; onde

$$r(p, a, b) = \begin{pmatrix} a - (I(p, a) - b)\,\dot\sigma(\beta)p + (I(p, a) - b)\,\dot\sigma(\beta)p) \\ a - (\sigma(I(p, a) - b)\,\dot\sigma(\beta)p \end{pmatrix}$$

A segunda camada C : 2 → 1



representa a função parametrizada

$$Eu: R\times R \longrightarrow R; \quad (q, b) \mapsto \sigma(qb+ qb+ q).$$

que por sua vez possui funções de atualização e solicitação

$$U: R\times R\times R \longrightarrow R;$$

$$(q, b, c) \mapsto \begin{pmatrix} q - \varepsilon(I(q, b) - c)\,\dot{\sigma}(qb + qb + q)bq - \varepsilon(I(q, b) - c)\,\dot{\sigma}(qb + qb + q) \end{pmatrix}$$

$$r: R\times R\times R \longrightarrow R;$$

$$(q, b, c) \mapsto \begin{pmatrix} b - (I(q, b) - c)\,\dot{\sigma}(qb + qb + q)qb - (I(q, b) - c)\,\dot{\sigma}(qb + qb + q)q \end{pmatrix}$$

Assim, as camadas mapeiam respectivamente os alunos (R, I, U, r) e (R, I, U, r).

A funcionalidade diz que podemos recuperar compostos U e ras U = U∗ U e r = r∗ r. Por exemplo, podemos verificar se isso é verdade para a primeira coordenada p:

$$\begin{aligned} U * U(p, q, a, c) &= p - \varepsilon(I(p, a) - s(q, I(p, a), c))\,\dot{\sigma}(\beta)a \\ &= p - \varepsilon(J(q, I(p, a)) - c) \\ &\quad \dot{\sigma}(qI(p, a) + qI(p, a) + q)q\,\dot{\sigma}(\beta)a \\ &= U(p, q, a, c) \end{aligned}$$

Em particular, a funcionalidade descreve como fatorar as expressões para as entradas de U e rin de uma forma que nos permite paralelizar a computação e reutilizar expressões de forma eficiente.

VII. Discussão

Em resumo, neste artigo, desenvolvemos uma estrutura algébrica para descrever a composição de algoritmos de aprendizado supervisionado. Para isso, identificamos a noção de função de solicitação como a principal característica distintiva do aprendizado composicional. Essa função de solicitação nos permite construir dados de treinamento para todas as subpartes de um algoritmo de aprendizado composto a partir de dados de treinamento apenas para a entrada e a saída do algoritmo composto.

Essa perspectiva nos permite articular cuidadosamente a estrutura do algoritmo de retropropagação. Em particular, vemos que:

- Uma função de ativação σ define um functor de arquiteturas de redes neurais para funções parametrizadas.
- Um tamanho de passo ε e uma função de erro e definem um functor de funções parametrizadas para algoritmos de aprendizado supervisionado.
- A função de atualização para o algoritmo de aprendizagem definido por este functor é especificada pela descida do gradiente.
- A função de solicitação para o algoritmo de aprendizado definido por este functor é especificada por retropropagação.
- A estrutura bimonoide na categoria de algoritmos de aprendizagem nos permite entender redes neurais, incluindo variantes como as convolucionais, geradas a partir de três algoritmos básicos.
- Redes neurais fornecem uma linguagem composicional simples para especificar algoritmos de aprendizagem.
- A composição dos alunos, juntamente com o fato de que os gradientes são mais rápidos de calcular para espaços de dimensões inferiores, expressa a velocidade de aprendizagem proporcionada pela retropropagação.

Encerramos com algumas observações sobre outras orientações.

A. Funções de erro mais gerais

Para aplicar nosso teorema principal e, portanto, entender a retropropagação como um functor, precisamos que certas derivadas da função de erro escolhida sejam invertíveis. Algumas funções de erro comumente usadas, no entanto, não obedecem exatamente a essas condições. Por exemplo, a entropia cruzada é uma função de erro semelhante ao erro quadrático, mas frequentemente leva a uma convergência mais rápida. A entropia cruzada é dada por

$$e(x, y) = y \ln x + (1 - y) \ln(1 - x).$$

Isso não fornece um exemplo do teorema principal, pois a derivada não é definida quando x = 0, 1.

No entanto, está bem próximo de um exemplo. Há duas maneiras pelas quais o método prático difere da nossa teoria. Primeiro, em vez de simplesmente somar o erro para chegar ao nosso erro total E, o método usual de usar entropia cruzada calcula a média, resultando na função

$$E(p, a, b) = \frac{1}{n} \sum_{j=1}^{n} e(I(p, a), b)$$

onde n é a dimensão do espaço vetorial do contradomínio B. Isso é bastante simples de modelar, e mostramos como fazer isso incorporando uma variável extra α em nossa generalização do teorema principal no Apêndice A da versão estendida [10].

O segundo é mais sutil. Quando x, 0, 1, a entropia cruzada tem a derivada

$$\frac{\partial e}{\partial x}(z, y) = \frac{y - z}{z(1 - z)}.$$

Isto é invertível para todos os z, 0, 1. Na prática, consideramos (i) dados de treinamento (a, b) tais que 0 ≤ a, b≤ 1 para todos os i, j, bem como (ii) I(p, a) tal que isto implica 0 < I(p, a) < 1 para todos os k, assumindo que começamos com um parâmetro inicial adequado p e um tamanho de passo ε suficientemente pequeno. Neste caso, (z, −) é invertível em todos os pontos relevantes, e assim podemos definir funções de solicitação.

De fato, neste caso a função de solicitação é

$$r(p, a, b) = a - a(1 - a) \sum_{j} \frac{Eu(p, a) - b\, Eu(p, a)}{(1 - I(p, a))} \frac{\partial I}{\partial a}(p, a),$$

enquanto a função de atualização é a regra de atualização padrão para descida de gradiente em relação à entropia cruzada.

$$U(p, a, b) = p - \varepsilon \sum_{j} \frac{Eu(p, a) - b\, Eu(p, a)}{(1 - I(p, a))} \frac{\partial I}{\partial p}(p, a).$$

Há trabalho a ser feito na generalização do teorema principal para acomodar funções de erro, como a entropia cruzada, que não possuem derivadas em pontos isolados. Independentemente disso, observe que, embora neste caso não seja simples declarar a retropropagação como um functor de Para, nossa análise ainda esclarece a natureza composicional do algoritmo de aprendizagem.

B. Algoritmos de aprendizagem em rede generalizados

A categoria Learn contém muito mais morfismos do que aqueles nas imagens de Para sob os functores de descida de gradiente/retropropagação L. De fato, Learn não exige que definamos nossas funções de atualização e solicitação usando derivadas. Isso mostra que podemos introduzir elementos muito mais gerais do que as redes neurais usuais em algoritmos de aprendizado de máquina e ainda usar um método modular, semelhante à retropropagação, para aprender.

Como seriam algoritmos de aprendizagem mais gerais? Como os espaços de entrada/saída não precisam ser euclidianos, poderíamos escolher partes do nosso algoritmo para aprender funções que são restringidas a obedecer a certas simetrias, como periodicidade, ou, equivalentemente, definidas em um toro. De fato, o aprendizado sobre variedades equipadas com alguma estrutura diferenciável é um campo de estudo ativo [5]. Também poderíamos aprender funções não lineares, como rotações, ou encontrar uma maneira de parametrizar arquiteturas de rede.

Há uma vantagem clara em usar a descida do gradiente: ela fornece um argumento heurístico de que o algoritmo de aprendizagem se atualiza no sentido de reduzir alguma função, que podemos interpretar como o erro. Isso ajuda a orientar a construção de uma rede neural. Observe, no entanto, que a categoria Aprender não vê nada dessa estrutura; ela reside nos functores L. Assim, Aprender nos permite construir algoritmos de aprendizagem que variam a noção de erro ao longo da rede.

Por fim, as redes neurais são úteis porque fornecem uma linguagem combinatória simples para especificar algoritmos de aprendizado supervisionado. Na Seção V, vimos que esse fato pode ser expresso em termos categóricos da seguinte forma: as redes neurais são úteis porque são a linguagem gerada, utilizando a gramática de categorias monoidais simétricas, a partir de apenas alguns aprendizes (multiplicação escalar, viés, funtor de ativação, multiplicação monoide e comultiplicação). A escolha de outras primitivas poderia fornecer uma nova linguagem igualmente simples para especificar algoritmos de aprendizado adaptados a uma aplicação escolhida.

C. Uma bicategoria de alunos

Atualmente, as abordagens para ajustar hiperparâmetros de uma rede neural são bastante ad hoc. Um desses hiperparâmetros é a arquitetura da própria rede. Quantas camadas a rede neural ideal possui para um determinado problema?

E quantos nós devem existir em cada camada? Uma bicategoria é uma generalização de uma categoria na qual também existem morfismos bidimensionais conectando os morfismos usuais. Aprendizes formam naturalmente uma bicategoria. De fato, nossa definição de equivalência de aprendizes usa implicitamente essa estrutura; equivalência é apenas isomorfismo para a seguinte noção de 2-morfismo.

Definição VII.1. Um 2-morfismo $f : (P, I, U, r) \to (Q, J, V, s)$ de aprendizes é uma função $f : P \to Q$ tal que $J(f (p), a) = I(p, a)$, $V(f (p), a, b) = f (U(p, a, b))$, e $s(f (p), a, b) = r(p, a, b)$.

Da mesma forma, Para e IDAG também são naturalmente bicategorias. Trabalhar nesse cenário bicategorial fornece linguagem para relacionar diferentes funções parametrizadas e arquiteturas de redes neurais. Esses morfismos superiores podem codificar ideias como a expansão estruturada de redes, adicionando neurônios ou camadas adicionais.

D. Aprendizes, lentes e jogos abertos

Definimos a categoria de aprendizes para modelar a troca de informações entre unidades de aprendizagem individuais e como isso cria um aprendiz maior e composto. Da mesma forma, a teoria das categorias tem sido usada para modelar abstratamente linguagens de programação bidirecionais e bancos de dados, utilizando várias noções de lentes e jogos microeconômicos interativos, resultando na noção de um jogo aberto.

Essas análises categóricas revelam semelhanças estruturais marcantes entre esses três assuntos, unificadas pela ideia de que, em essência, eles estudam como os agentes trocam e respondem a informações. De fato, lentes assimétricas são simplesmente aprendizes com espaços de estados triviais, e os próprios aprendizes são jogos abertos que obedecem a uma determinada condição única de melhor resposta. Escrevendo Lente e Jogo para as respectivas categorias (definidas em [14] e [11]), isso resulta em embeddings

$$\mathrm{Lente}^{\mathrm{op}} \to \mathrm{Aprender}^{\mathrm{op}} \to \mathrm{Jogo}.$$

Por meio desses functores, a função de implementação corresponde à função "get" de uma lente e à função "play" de um jogo aberto, enquanto a função de solicitação corresponde às funções "put" e "coplay". A função de atualização do aprendiz corresponde à função de atualização da estratégia, conhecida como função de melhor resposta, para o jogo aberto [9], [12]. Além disso, a categoria "Learn" também se insere em uma determinada categoria de lentes simétricas [9].

Lentes vêm com várias noções de "bom comportamento", que impõem condições de compatibilidade entre funções put e get. Até agora, no caso de aprendizes, não estabelecemos requisitos para que um algoritmo convirja para uma função f quando dados pares de treinamento suficientes (a, f(a)). Examinar a relação lente-aprendiz pode fornecer insights sobre como não apenas definir estruturas que aprendem, mas que aprendem bem.

Referências

[1] JC Baez, B. Fong e B. Pollard. Uma estrutura composicional para processos de Markov. Journal of Mathematical Physics, 57(3):033301, 2016. doi:10.1063/1.4941578. [2] JC Baez e J. Erbele Categorias em Controle. Teoria e Aplicação de Categorias, 30(24):836–881, 2015. [3] A. Bohannon, BC Pierce e JA Vaughan. Lentes relacionais: uma

Linguagem para visualizações atualizáveis. Anais do vigésimo quinto simpósio ACM SIGMOD-SIGACT-SIGART sobre Princípios de sistemas de banco de dados (PODS '06), pp. 338–347, 2006. doi:10.1145/1142351.1142399.

[4] F. Bonchi, P. Soboci ´nski e F. Zanasi Álgebras de Hopf Interagindo. Journal of Pure and Applied Algebra, 221(1):144–184, 2017.

doi:10.1016/j.jpaa.2016.06.002.

[5] MM Bronstein, J. Bruna, Y. LeCun, A. Szlam, P. Vandergheynst. Aprendizado profundo geométrico: indo além dos dados euclidianos. Revista IEEE Signal Processing, 34(4):18–42, 2017.

doi:10.1109/MSP.2017.2693418.

[6] B. Coecke e A. Kissinger. Representando Processos Quânticos: Um Primeiro Curso em Teoria Quântica e Raciocínio Diagramático. Cambridge University Press, 2017.

[7] M. Fiore e M. Devesas Campos. A Álgebra dos Gráficos Acíclicos Dirigidos. In: B. Coecke, L. Ong, P. Panangaden (eds.)

Computação, Lógica, Jogos e Fundamentos Quânticos. Os Muitos

Facetas de Samson Abramsky. Notas de aula em Ciência da Computação, vol. 7860. Springer, Berlim, Heidelberg. arXiv:1303.0376.

[8] B. Fong. A Álgebra de Sistemas Abertos e Interconectados. Tese de doutorado, Universidade de Oxford, 2016. arXiv:1609.05382. [9] B. Fong e M. Johnson. Lentes e Aprendizes. A ser publicado

em Anais do Oitavo Workshop Internacional sobre Transformações Bidirecionais (Bx2019). arXiv:1903.03671

[10] B. Fong, DI Spivak, R. Tuy´eras. Backprop como Functor: Uma composição

perspectiva situacional sobre aprendizagem supervisionada. (Versão estendida.) arXiv:1711.10455

[11] N. Ghani, J. Hedges, V. Winschel, P. Zahn. Composicional

Teoria dos Jogos. Anais do 33º Simpósio Anual ACM/IEEE sobre Lógica em Ciência da Computação (LICS '18), pp. 472–481, 2018.

doi:10.1145/3209108.3209165.

[12] J. Hedges. De aprendizes abertos a jogos abertos. Pré-impressão.

arXiv:1902.08666 [13] Y. Le Cun e T. Bengio. Redes convolucionais para imagens,

discurso e séries temporais. Em MA Arbib (Ed.), O manual de teoria do cérebro e redes neurais, pp. 255–258, MIT Press, 1995.

[14] M. Johnson e RD Rosebrugh. Vãos de lentes. Workshops EDBT/ICDT, pp.112–118, 2014. [15] A. Joyal e R. Street. A geometria do cálculo tensorial I. Advances in Mathematics 88(1):55–112, 1991.

doi:10.1016/0001-8708(91)90003-P.

[16] S. Mac Lane. Categorias para o matemático trabalhador, Springer, Berlim, 1998. [17] A. Mahendran e A. Vedaldi. Compreendendo a representação de imagens profundas

representações invertendo-as. Anais da conferência do IEEE sobre visão computacional e reconhecimento de padrões, pp. 5188–5196, 2015.

[18] MA Nielsen. Redes Neurais e Aprendizado Profundo. Determinação

Press, 2015. http://neuralnetworksanddeeplearning.com [19] D. Vagner, D. Spivak. E. Lerman. Álgebras de sistemas dinâmicos abertos na operação de diagramas de fiação. arXiv:1408.1598.

Apêndice

A. Prova da Proposição II.4

Demonstração da Proposição II.4. Isto decorre da verificação rotineira dos axiomas; diremos algumas palavras sobre cada caso. Observe que os argumentos abaixo são independentes da escolha do representante da classe de equivalência do aprendiz.

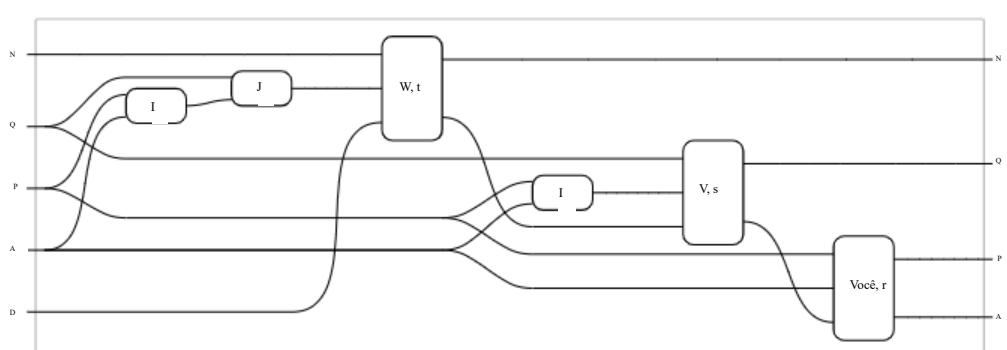
a) Identidades: Os axiomas de identidade são facilmente verificados. Por exemplo, para verificar a identidade à esquerda, vemos que $(P, I, U, r) * (R, id, !, \pi)$ é dado por $P \times RP$, $I(p, id(a)) = I(p, a)$, $(U * !)(p, a, b) = U(p, a, \pi(I(p, a), b)) = U(p, a, b)$, e $(r * \pi)(p, a, b) = r(p, a, \pi(I(p, a), b)) = r(p, a, b)$.

b) Associatividade: O axioma da associatividade é o que exige que nossos morfismos em Learn sejam classes de equivalência de aprendizes, e não simplesmente os próprios aprendizes: a composição de aprendizes não é associativa de imediato. De fato, isso ocorre porque produtos de conjuntos não são associativos de imediato: temos apenas isomorfismos $(P \times Q) \times N P \times (Q \times N)$ de conjuntos, não igualdade. Reconhecendo isso, a associatividade é simples de provar. Sejam $(P, I, U, r): A \to B$, $(Q, J, V, s): B \to C$ e $(N, K, W, t): C \to D$ aprendizes. O item mais complexo a ser verificado é a associatividade da função de atualização-solicitação pareada. O cálculo mostra

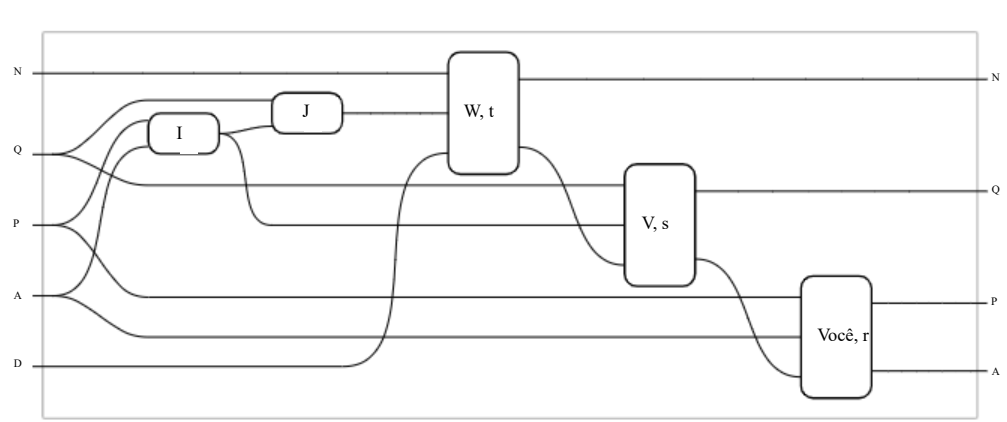
$$
\begin{aligned}
&(U * V) * W \\
&= \left(U\left(p, a, s(q, I(p, a), \gamma)\right), V\left(q, I(p, a), \gamma\right), C\left(n, J(q, I(p, a)), d\right)\right) \\
&= U * (V * W)
\end{aligned}
$$

onde $\gamma = t\left(n, J(q, I(p, a)), d\right)$.

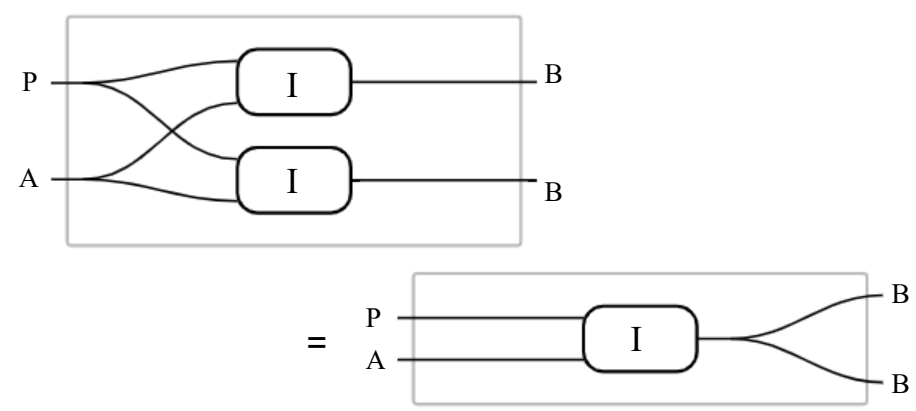
Esta igualdade é mais fácil de analisar usando diagramas de strings. O composto $(U * V) * W$ é dado pelo diagrama



enquanto o composto $U * (V * W)$ é dado por



Para provar que esses dois diagramas representam a mesma função, observe que a função $(I(p, a), I(p, a)) : P \times A \to B \times B$ pode ser desenhada das duas maneiras a seguir:



Essa igualdade e a associatividade do mapa diagonal implicam a igualdade dos dois diagramas anteriores e, portanto, a associatividade dos compostos de atualização e solicitação.

c) Monoidalidade: É fácil verificar se o acima é um produto monoidal, com unidade dada pelo conjunto de um elemento $\{*\}$.

De fato, note que agora mostramos que Learn é uma categoria. Existe um functor da categoria Conjunto de conjuntos e funções para Learn. Este functor mapeia cada conjunto para si mesmo, e cada função $f : A \to B$ para a função trivialmente parametrizada $f : R \times A \to B$. Note que (Set, ×) é uma categoria monoidal, e sejam α, ρ e λ respectivamente o associador, o unitor direito e o unitor esquerdo para (Set, ×). As imagens destes mapas sob este functor de parametrização trivial $(\overline{\cdot})$, escritos α, ρ e λ, são os mapas de estrutura correspondentes para (Learn, ‖) como uma categoria monoidal simétrica.

A naturalidade desses mapas, bem como os axiomas de uma categoria monoidal simétrica, seguem então de forma direta os fatos correspondentes em Set.

Assim, definimos uma categoria monoidal simétrica. □

B. Prova do Teorema III.2

Provamos um teorema um pouco mais geral, incorporando uma variável extra α para melhor descrever o caso de entropia cruzada, conforme discutido na Seção VII-A.

Teorema A.1 (Generalização do Teorema III.2). Fixe $\varepsilon > 0$, $\alpha : N \to R$, e $e(x, y) : R \times R \to R$ diferenciáveis tais que

$\frac{\partial e}{\partial x}(z, -) : R \to R$ é invertível para cada $z \in R$.

Então podemos definir um functor monoidal simétrico, injetivo sobre objetos e fiel

$$L: \text{Para} \longrightarrow \text{Aprender}$$

que envia cada função parametrizada $I : P \times A \to B$ para o aluno (P, I, U, r) definida por

$$U(p, a, b) \coloneqq p - \varepsilon \nabla E(p, a, b)$$

e

$$r(p, a, b) \coloneqq f\left(\frac{1}{\alpha}\nabla E(p, a, b)\right),$$

onde fis aplicação componente a componente do inverso para $\frac{\partial e}{\partial x}(a, -)$ para cada i, e

$$E(p, a, b) \coloneqq \alpha \sum_j e(I(p, a), b).$$

Demonstração. O functor L é, por definição, injetivo em objetos. Como I mapeia para (P, I, U, r), o functor L é injetivo em morfismos e, portanto, fornecerá um functor fiel.

Sejam I : P × R → Rand J : Q × R → R funções parametrizadas. Mostramos que a composição de suas imagens é igual à imagem de sua composição.

a) Funções de atualização: Por definição, o composto das funções de atualização de I e J é dado por

$$\begin{aligned}
&(U * U)(p, q, a, c) \\
&= \left(U(p, a, r(q, I(p, a), c)), U(q, I(p, a), c)\right) \\
&= \left(p - \varepsilon\nabla E(p, a, r(q, I(p, a), c)), q - \varepsilon\nabla E(q, I(p, a), c)\right),
\end{aligned}$$

enquanto a função de atualização do composto I ∗ J é

$$U(p, q, a, c) = \left(p - \varepsilon\nabla E(p, q, a, c), q - \varepsilon\nabla E(p, q, a, c)\right).$$

Para mostrar que estas são iguais, devemos mostrar que as seguintes equações são válidas

$$\nabla E(p, a, r(q, I(p, a), c)) = \nabla E(p, q, a, c) \quad (1)$$

$$\nabla E(q, I(p, a), c) = \nabla E(p, q, a, c). \quad (2)$$

Primeiro, consideramos a Equação 1:

$$\begin{aligned}
&\nabla E(p, a, r(q, I(p, a), c)) \\
&= \nabla\alpha \sum_i e\left(Eu(p, a), r(q, I(p, a), c)\right) && \text{(def E)} \\
&= \left(\alpha \sum_i \frac{\partial e}{\partial x}\left(Eu(p, a), r(q, I(p, a), c)\right) \frac{\partial I}{\partial p}\cdot(p, uma)_\ell\right) && \text{(definição } \nabla) \\
&= \left(\alpha \sum_i \frac{\partial e}{\partial x}\left(Eu(p, a), f\left(\frac{1}{\alpha}\nabla E(q, I(p, a), c)\right)\right)_i \frac{\partial I}{\partial p}\cdot(p, uma)_\ell\right) && \text{(definição r)} \\
&= \left(\alpha \sum_i \frac{1}{\alpha}\left(\nabla E(q, I(p, a), c)\right)_i \frac{\partial I}{\partial p}\cdot(p, uma)_\ell\right) && \text{(definição f)} \\
&= \left(\alpha \sum_i \frac{1}{\alpha}\left(\nabla\alpha \sum_j e(J(q, I(p, a)), c)\right)_i \frac{\partial I}{\partial p}\cdot(p, uma)_\ell\right) && \text{(def E)} \\
&= \left(\sum_i \alpha \sum_j \frac{\partial e}{\partial x}(J(q, I(p, a)), c) \frac{\partial J}{\partial b}(q, I(p, a)) \frac{\partial I}{\partial p}\cdot(p, uma)_\ell\right) && \text{(definição } \nabla) \\
&= \left(\alpha \sum_j \frac{\partial e}{\partial x}(J(q, I(p, a)), c) \frac{\partial J}{\partial p}(q, I(p, a))\right)_\ell && \text{(regra da cadeia)} \\
&= \nabla\alpha \sum_j e(J(q, I(p, a)), c) && \text{(definição } \nabla) \\
&= \nabla E(p, q, a, c) && \text{(def E)}
\end{aligned}$$

Observe a mudança para o raciocínio coordenado, em que f é definido como o inverso de ∂e, e o uso da regra da cadeia. Aqui, i, j e ℓ são indexados sobre as dimensões de B, C e P, respectivamente.

A equação 2 simplesmente decorre da definição da função de erro; não precisamos nem mesmo tomar derivadas:

$$E(q, I(p, a), c) = \alpha \sum_i e\left(J(q, I(p, a)), c\right) = E(p, q, a, c)$$

Assim, mostramos U∗ U= U, como desejado.

b) Funções de solicitação: Devemos provar que a seguinte equação é válida:

$$r(p, a, r(q, I(p, a), c)) = r(p, q, a, c)$$

Isso ocorre devido à regra da cadeia, exatamente da mesma maneira que para atualizar p, mas trocando os papéis de a e p na prova da Equação 1:

$$\begin{aligned}
r(p, a, r(q, I(p, a), c)) &= f\left(\frac{1}{\alpha}\nabla E(p, a, r(q, I(p, a), c))\right) \\
&= f\left(\frac{1}{\alpha}\nabla E(p, q, a, c)\right) \\
&= r(p, q, a, c)
\end{aligned}$$

c) Identidades: A identidade no objeto A na categoria de funções parametrizadas é a projeção id: R × A → A. A imagem de id tem uma função de atualização trivial, visto que o espaço de parâmetros é trivial. A função de solicitação é dada por

$$r(0, a, b) = f\left(\nabla\left(\alpha \sum_i e(a, b)\right)\right) = \left(f\left(\frac{\partial e}{\partial a}(a, b)\right)\right)_i = b.$$

Este é exatamente o mapa de identidade (R, id, !, π) em Learn.

d) Estrutura monoidal: O functor L é um functor monoidal. Ou seja, o aprendiz dado pelo produto monoidal de funções parametrizadas é igual ao produto monoidal dos aprendizes dados por essas mesmas funções, até os isomorfismos padrão R × R R. Para ver que isso é verdade, suponha que temos funções parametrizadas I : P × A → B e J : Q × C → D. Seu tensor é I ∥ J : (P × Q) × (A × C) → B × D. Note que E(p, q, a, c, b, d) = E(p, a, b) + E(q, c, d). Assim, a função de atualização de seu tensor é dada por

$$\begin{aligned}
&U(p, q, a, c, b, d) \\
&= \left(p - \varepsilon\nabla E(p, q, a, c, b, d)), q - \varepsilon\nabla E(p, q, a, c, b, d)\right) \\
&= \left(p - \varepsilon\nabla E(p, a, b)), q - \varepsilon\nabla E(q, c, d)\right) \\
&= \left(U(p, a, b), U(q, c, d)\right)
\end{aligned}$$

e da mesma forma a função de solicitação é

$$
\begin{aligned}
& r(p, q, a, c, b, d) \\
= f & \left(\alpha\nabla(a,c) E(p, q, a, c, b, d) \right) \\
= f & \left(\frac{1}{\alpha\nabla(a,c)\alpha B\times D} \left(\sum_i e(I(p, a), b) + \sum_j e(J(q, c), d) \right) \right) \\
= & \left(f\left(\alpha\nabla a E(p, a, b) \right), f\left(\alpha\nabla c E(q, c, d) \right) \right) \\
= & \left(r(p, a, b), r(q, c, d) \right)
\end{aligned}
$$

Portanto, a imagem do tensor é o tensor da imagem.

C. Contexto da teoria das categorias

1) Categorias monoidais simétricas: Uma categoria simétrica

A categoria monoidal é uma configuração para composição de linguagens diagramáticas em estilo de rede, como redes neurais. Uma propriedade é um tipo particularmente simples de categoria monoidal simétrica estrita.

Primeiro, vamos definir uma categoria. Especificamos uma categoria C usando os dados:

- uma coleção X cujos elementos são chamados objetos.
- para cada par (A, B) de objetos, um conjunto [A, B] cujos elementos são chamados morfismos.
- para cada triplo (A, B, C) de objetos, uma função [A, B]× [B, C] → [A, C] chamamos de regra de composição, e onde escrevemos (f, g) 7 → f ; g.

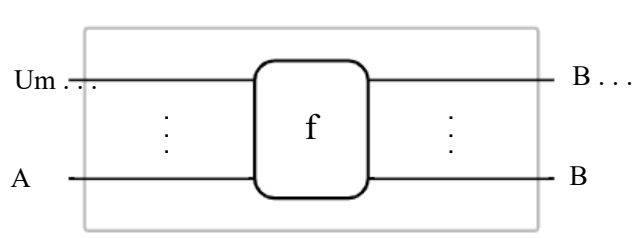
Esses dados estão sujeitos aos axiomas

- identidade: para todos os objetos A existe id∈ [A, A] tal que para todos os f ∈ [A, B] e g ∈ [B, A] temos id; f = f e g; id= g.
- associatividade: para todos os f ∈ [A, B], g ∈ [B, C] e h ∈ [C, D] temos (f ; g); h = f ; (g; h).

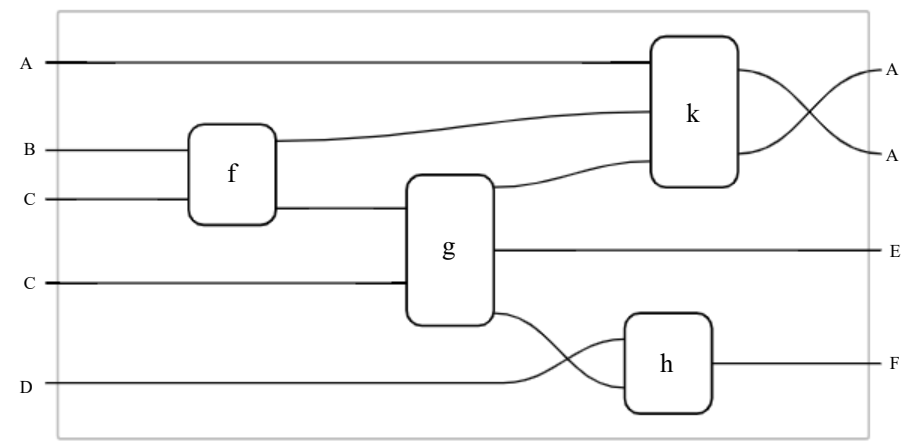
O principal objeto de nosso interesse, no entanto, é um tipo específico de categoria, conhecida como categoria monoidal simétrica. Para uma categoria monoidal simétrica C, precisamos ainda dos dados:

- para cada par (A, B) de objetos, outro objeto A ⊗ B em X.
- para cada quádruplo (A, B, C, D) de objetos, uma função [A, B] × [C, D] → [A ⊗ C, B ⊗ D] chamada de produto monoidal.

Usando esses dados, podemos desenhar redes. Pensamos nos objetos como sendo vários tipos de fios, e um morfismo f em [A⊗ · · · ⊗ A, B⊗ · · · ⊗ B] como uma caixa com fios do tipo A à esquerda e fios do tipo B à direita. Aqui estão algumas imagens.



Conectando fios do mesmo tipo, podemos desenhar figuras mais complexas. Por exemplo:



O ponto-chave de uma rede é que qualquer imagem desse tipo deve ter uma interpretação inequívoca como um morfismo. O uso de diagramas de strings para representar morfismos em uma categoria monoidal é formalizado em [15].

Para formar o que é conhecido como uma categoria monoidal simétrica estrita, os dados acima devem obedecer a axiomas adicionais que garantam que eles capturem a intuição acima de se comportar como uma rede. Esses axiomas são

- intercâmbio: para todos os f ∈ [A, B], g ∈ [B, C], h ∈ [D, E], k ∈ [E, F] temos (f ; g) ⊗ (h; k) = (f ⊗ h); (g ⊗ k).
- identidade monoidal: existe um objeto I tal que I ⊗ A = A = A ⊗ I.
- associatividade monoidal: para todos os objetos A, B, C temos (A ⊗ B) ⊗ C = A ⊗ (B ⊗ C).
- simetria: para todos os pares de objetos A, B temos morfismos σ∈ [A ⊗ B, B ⊗ A] tais que σ; σ= id, e que para todo f ∈ [A, C], g ∈ [B, D] temos (f ⊗ g); σ= σ; (f ⊗ g).

De forma mais geral, categorias monoidais simétricas exigem que esses axiomas sejam verdadeiros apenas até o isomorfismo natural. Mais detalhes podem ser encontrados em [16].

Exemplo A.2. Um exemplo de categoria monoidal simétrica é (Conjunto, ×), onde nossos objetos são um conjunto de cada cardinalidade, e os morfismos são funções entre eles. O produto monoidal é dado pelo produto cartesiano dos conjuntos.

Exemplo A.3. Outro exemplo de uma categoria monoidal simétrica é (FVect, ⊕), onde nossos objetos são espaços vetoriais de dimensão finita, morfismos são aplicações lineares e o produto monoidal é dado pela soma direta de espaços vetoriais.

2) Functores: Um functor é uma maneira de reinterpretar uma categoria em outra, preservando a estrutura algébrica. Em outras palavras, um functor é a noção de aplicação que preserva a estrutura para categorias, em analogia com transformações lineares como aplicações que preservam a estrutura para espaços vetoriais, e homomorfismos de grupo como aplicações que preservam a estrutura para grupos.

Formalmente, dadas as categorias C, D, um functor F : C → D envia cada objeto A de C para um objeto FA de D, cada

morfismo $f : A \to B$ em C para um morfismo $F f : FA \to FB$ em D, tal que $F1 = 1$ e $F f ; Fg = F(f ; g)$.

Um functor entre categorias monoidais simétricas é um functor monoidal simétrico se $FI= I$, onde I é a unidade monoidal para a categoria relevante, e se existem isomorfismos $F(A\otimes B)$ $FA\otimes FB$ naturais nos objetos A, B ou C. Dizemos que o functor é um functor monoidal simétrico estrito se esses isomorfismos são de fato igualdades.

Também dizemos que um functor é fiel se $F f = Fg$ somente quando $f = g$, e injetivo em objetos se o mapa de objetos de C para objetos de D for injetivo.

3) Bimonoides: Um bimonoide em uma categoria monoidal simétrica é um objeto A com morfismos que obedecem a certos axiomas. Esses morfismos têm nomes e tipos:

multiplicação unidade

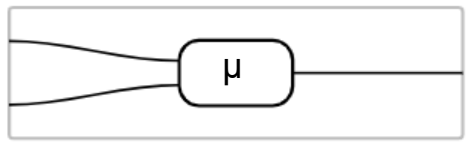
$\mu : A \otimes A \to A$ $\varrho : I \to A$

contagem de comultiplicação

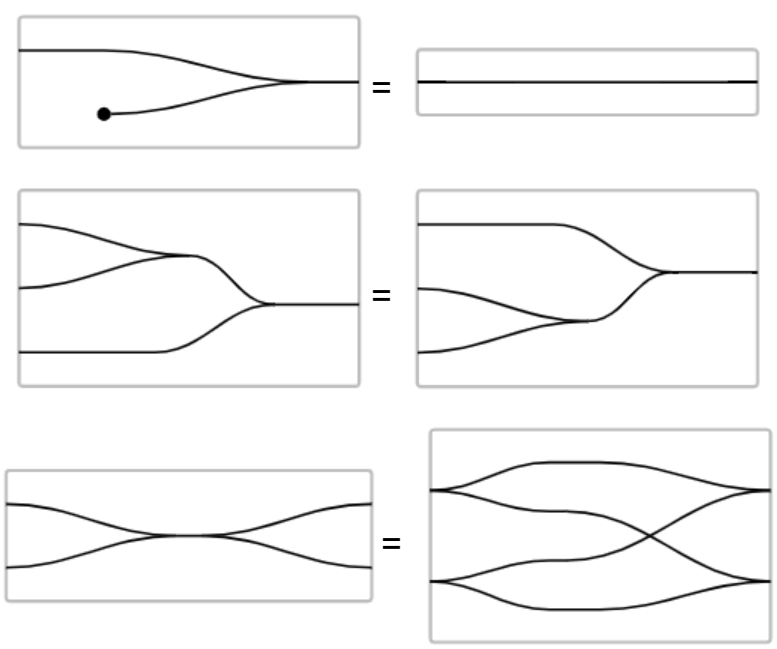
$\delta : A \to A \otimes A$ $\nu : A \to I$

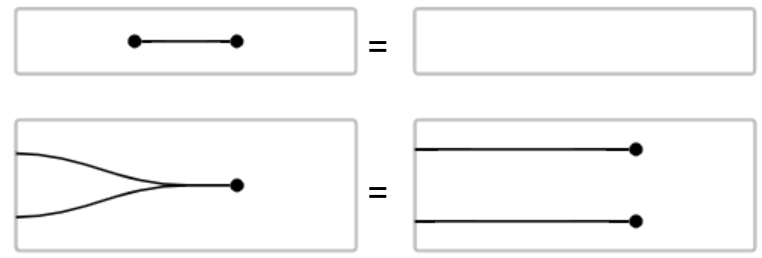
Observe que esses diagramas são representações informais, mas úteis, desses morfismos. Mais formalmente, por exemplo, o diagrama

para a multiplicação μ é uma abreviação para o diagrama de string



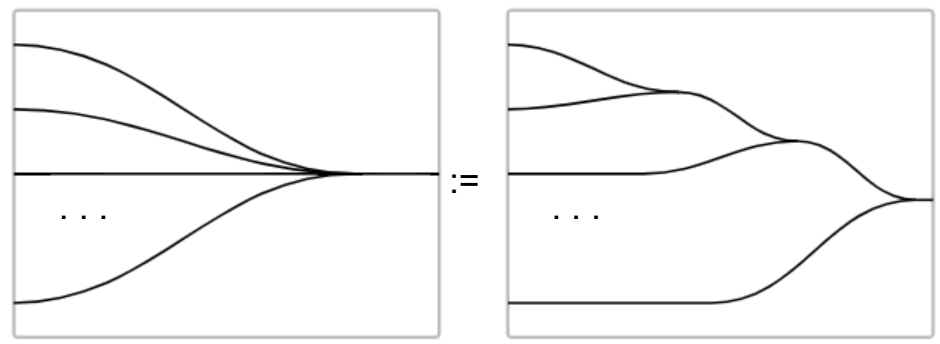
Esses morfismos devem obedecer aos axiomas:





e suas imagens espelhadas.

Observe que o segundo axioma acima, chamado associatividade, implica que todos os mapas com contradomínio 1 construídos usando apenas produtos da multiplicação e do mapa identidade são iguais. Portanto, é conveniente, e não causa confusão, definir a seguinte notação:



Definimos a notação de imagem espelhada para comultiplicações iteradas.

Esses morfismos, e os axiomas aos quais obedecem, permitem a construção de diagramas de rede. Primeiramente, os morfismos μ, ϱ, δ e ν, respectivamente, dão interpretações para a fusão em pares, inicialização, divisão em pares e exclusão de arestas. Os axiomas de associatividade e coassociatividade, por exemplo, dão uma interpretação única para a fusão e a divisão n-árias, conforme descrito acima.

Exemplo A.4. Cada objeto em FVect pode ser equipado com a estrutura de um bimonoide. De fato, dado um espaço vetorial V, a multiplicação $\mu : V \oplus V \to V$ leva um par (u, v) para u+v, a unidade $\varrho : 0 \to V$ mapeia o elemento único 0 do espaço vetorial 0-dimensional para o vetor zero em V, a comultiplicação $\delta : V \to V\oplus V$ mapeia v para (v, v), e a contagem $\nu : V \to 0$ mapeia todo vetor $v \in V$ para zero. É álgebra linear padrão verificar se esses mapas obedecem aos axiomas bimonoides; veja [2], [4] para detalhes.